

RELATÓRIO TÉCNICO: SIMULADOR DE GERENCIADOR DE MEMÓRIA VIRTUAL

Disciplina: Sistemas Operacionais

Instituição: PUC Minas

Grupo: Christian Augusto Soares Diniz, João Victor Batista Carneiro, Thulio Hiroshi Fernandes Funayama, Rafael Araujo Pereira, Benedito Medeiros Salomão

Link do Repositório: https://github.com/Kurisnya/SO--Trabalho_Pratico_2.git

Introdução e Fundamentação Teórica

O gerenciamento de memória por paginação por demanda é um dos pilares dos sistemas operacionais modernos, permitindo que o espaço de endereçamento virtual de um processo seja maior do que a memória física real instalada no sistema.

Este relatório documenta o desenvolvimento de um simulador de gerenciador de memória virtual desenvolvido na linguagem C. O sistema opera traduzindo endereços lógicos de 32 bits (onde apenas os 16 bits menos significativos são considerados) em endereços físicos correspondentes. O espaço virtual simulado compreende bytes, divididos em 256 páginas de 256 bytes cada. Em contrapartida, a memória física é limitada a 32.768 bytes, contendo 128 quadros (*frames*) de 256 bytes.

Para otimizar o tempo de tradução, o projeto integra dois componentes fundamentais estudados na teoria de sistemas operacionais:

1. **Translation Lookaside Buffer (TLB):** Um cache de hardware rápido com capacidade de 16 entradas que armazena as traduções de páginas para quadros utilizadas mais recentemente, operando sob a política de substituição FIFO (First-In, First-Out).
2. **Tabela de Páginas (Page Table):** Uma estrutura de mapeamento indexada de 256 entradas utilizada quando ocorre uma omissão no TLB (*TLB Miss*).

Devido à restrição física de espaço (128 quadros para 256 páginas possíveis), o simulador adota o algoritmo de envelhecimento (Aging Algorithm) para gerenciar a paginação por demanda e a substituição de páginas vítimas quando a memória física se encontra saturada.

Descrição da Arquitetura e Estruturas de Dados

O simulador foi estruturado de forma modularizada seguindo estritamente o template proposto:

1. Tabela de Páginas (page_table.c)

Implementada a partir de uma estrutura de dados (struct) denominada `page_table_entry_t` mapeada em um vetor estático de tamanho `PAGE_TABLE_SIZE` (256 entradas). Cada entrada armazena:

- `int frame`: O número do quadro físico associado.
- `int valid`: Bit de validade (1 se residente em memória, 0 se inválido).
- `int reference_bit`: Bit de referência () modificado a cada acesso.
- `unsigned char aging_counter`: Registrador de deslocamento de 8 bits para controle do envelhecimento histórico de acessos.

2. Cache TLB (tlb.c)

Estruturado através do tipo `tlb_entry_t` em um array estático de tamanho `TLB_SIZE` (16 entradas). Mapeia de forma direta os campos de identificação de página, número de quadro e bit de validade interna. O controle da fila FIFO é mantido através de uma variável inteira estática `fifo_next`, atuando como um ponteiro circular via aritmética modular.

3. Memória Física e de Retaguarda (memory.c)

A memória RAM simulada é representada pela matriz bidimensional `physical_memory[NUM_FRAMES][FRAME_SIZE]` de caracteres sinalizados (`signed char`), totalizando os 128 quadros de 256 bytes. O arquivo `BACKING_STORE.bin` atua como o armazenamento em massa secundário (disco), acessado de forma randômica por demanda de faltas de página. A associação reversa para fins de desalocação é controlada pelo vetor sequencial `frame_to_page [NUM_FRAMES]`.

Descrição da Implementação e Decisões de Projeto

O fluxo de execução do simulador segue uma lógica sequencial bem definida a cada endereço lido do arquivo de entrada:

1. **Extração de Componentes (Bitwise):** O endereço de 32 bits passa por máscaras e deslocamentos binários nativos. O deslocamento (*offset*) é isolado aplicando uma operação AND binária com a máscara 0xFF. O número da página é rotacionado em 8 bits para a direita ($\gg 8$) e igualmente mascarado com 0xFF.
2. **Consulta Hierárquica:** O identificador da página é submetido ao `tlb_lookup()`. Havendo sucesso (*TLB Hit*), o quadro é retornado imediatamente. Em caso de omissão (*TLB Miss*), recorre-se ao `page_table_lookup()`.
3. **Tratamento de Falta de Página (Page Fault):** Se a página consultada não possuir o bit de validade ativo na tabela, a rotina `handle_page_fault()` é disparada. O sistema busca por quadros livres via `find_free_frame()`. Se a memória estiver cheia, invoca-se o `select_victim_page()`, que varre as páginas ativas e escolhe aquela que possuir o menor valor numérico em seu `aging_counter`. A página escolhida sofre invalidação na tabela de páginas (`page_table_invalidate`) e expulsão do cache TLB (`tlb_remove`).
4. **I/O e Atualização:** A função posiciona o ponteiro do arquivo `BACKING_STORE.bin` na posição exata calculada por `page * FRAME_SIZE` através de `fseek()` e efetua a cópia em bloco de 256 bytes por meio de `fread()` diretamente para a memória física. O par é registrado na tabela de páginas, inserido no cache do TLB sob a regra FIFO e o endereço físico final é montado combinando o frame e o offset ($(\text{frame} \ll 8) \mid \text{offset}$).

Dificuldades Encontradas e Soluções

- **Inconsistência de Nomenclatura no Template:** Durante a compilação inicial da integração, o compilador acusou um erro de referência indefinida (*undefined reference*) para a função `tlb_invalidate`. Analisando os arquivos fontes, constatou-se que o

cabeçalho gerado pelo professor havia nomeado a função correspondente como `tlb_remove` dentro do arquivo `tlb.c`. A discrepância foi resolvida alinhando a chamada interna no arquivo `memory.c` para utilizar a nomenclatura exata `tlb_remove`.

- **Truncamento na Divisão Inteira:** No cálculo das taxas estatísticas, as primeiras execuções retornavam valores 0.000. Identificou-se que a linguagem C realiza a divisão estritamente inteira quando ambos os operandos são do tipo `int`. A falha foi solucionada aplicando a coerção explícita (`type casting`) dos contadores acumuladores para o tipo de dado de ponto flutuante `double` antes da realização da operação aritmética.

Testes Realizados e Análise dos Resultados

O simulador foi compilado com sucesso através das diretivas de otimização `-O2` e padrões rigorosos de checagem do compilador (`-Wall -Wextra -std=c11`), atendendo aos requisitos mínimos estabelecidos. Foram executadas as baterias completas de testes utilizando os dois cenários de carga de trabalho gerados pelo script utilitário do projeto.

Abaixo estão dispostos os dados consolidados obtidos a partir das execuções oficiais:

Tabela de Resultados Consolidados

Métrica Coletada	Cenário 1: Endereços Aleatórios (addresses_random.txt) PDF	Cenário 2: Endereços com Localidade (addresses_location.txt) PDF
Total de Endereços Traduzidos	10000	10000
Quantidade de Faltas de Página	4958	1164
Taxa de Faltas de Página (Page Fault Rate)	0.496	0.116
Quantidade de Acertos no TLB	558	7817
Taxa de Acertos no TLB (TLB Hit Rate)	0.056	0.782

Análise Crítica dos Resultados

A análise comparativa entre as duas cargas de trabalho demonstra de maneira clara o impacto prático dos princípios de **Localidade Temporal e Espacial** na computação.

No cenário de **Endereços Aleatórios** (`addresses_random.txt`), observa-se uma taxa de Page Fault substancialmente elevada e uma eficiência reduzida do cache TLB. Como as requisições de endereços são distribuídas de forma uniformemente dispersa pelo espaço virtual de memória, o algoritmo de envelhecimento (*Aging*) falha em prever acessos futuros, resultando em frequentes substituições e leituras repetidas do arquivo binário.

Por outro lado, no cenário de **Endereços com Localidade** (`addresses_location.txt`), ocorre o comportamento inverso. O cache TLB demonstra grande eficácia (elevação drástica no *TLB Hit Rate*), pois o programa executa laços de repetição ou acessa estruturas de dados sequenciais em posições de memória próximas em curtos intervalos de tempo. Da mesma forma, as faltas de páginas despencam acentuadamente, confirmando que a política de envelhecimento simulada consegue manter as páginas de trabalho ativas (*working set*) residentes de forma prolongada nos quadros físicos disponíveis.

Conclusão

O desenvolvimento deste projeto prático propiciou uma compreensão sólida e empírica dos mecanismos internos de baixo nível que regem a paginação por demanda e a tradução de endereços em um sistema computacional moderno. A implementação bem-sucedida das operações binárias por máscara, o controle de cache indexado circular (TLB FIFO) e a lógica de substituição global por relevância histórica (*Aging*) demonstraram o delicado equilíbrio existente entre custos de I/O em disco e a otimização de acessos à memória RAM. Os testes aplicados validaram não apenas a corretude matemática do simulador, mas ilustraram de forma visual o quão crítico é o comportamento e o padrão de escrita do código de software no aproveitamento das otimizações de hardware projetadas pelo sistema operacional.

Documentação do Uso de Inteligência Artificial (Metodologia SDD)

Conforme preconizado pelas diretrizes de transparência estabelecidas nas seções normativas do projeto, a equipe utilizou ferramentas de Inteligência Artificial seguindo rigorosamente o modelo de **Desenvolvimento Baseado em Especificações (Spec-Driven Development - SDD)**.

A IA atuou de maneira consultiva auxiliando no esclarecimento conceitual da manipulação de bits em C, na validação das estruturas lógicas do template e no isolamento de falhas estruturais durante os marcos do cronograma de desenvolvimento.

Abaixo constam documentados os prompts estruturados aplicados em cada fase lógica de desenvolvimento do simulador:

Prompt da Fase 2 – Passo 1: Extração de Componentes do Endereço Lógico

Contexto do Problema: Estou desenvolvendo um simulador de gerenciador de memória virtual em C para um trabalho prático de Sistemas Operacionais.

Requisitos Funcionais:

- O programa lê um endereço lógico que é um número inteiro de 32 bits.
- Deve considerar apenas os 16 bits menos significativos desse endereço.
- Desses 16 bits, os 8 bits mais significativos (bits 15 a 8) representam o número da página.
- Os 8 bits menos significativos (bits 7 a 0) representam o deslocamento (*offset*) da página.

Restrições de Implementação:

- A extração deve ser feita utilizando operações bitwise nativas da linguagem C (como >> e &).
- Gere uma função ou trecho de código limpo que receba `unsigned int logical_address` e preencha as variáveis `page_number` e `offset`.

Solicitação: Forneça o código em C para essa extração e explique brevemente o funcionamento das máscaras utilizadas.

Prompt da Fase 2 – Passo 2: Leitura do Arquivo de Retaguarda (BACKING_STORE.bin)

Contexto do Problema: Estou desenvolvendo a simulação da memória física e de retaguarda (BACKING_STORE) em C para um trabalho prático de Sistemas Operacionais.

Requisitos Funcionais:

- Preciso de uma função que receba o número de uma página virtual e o número de um quadro físico (*frame*).
- O simulador deve ler um bloco contíguo de 256 bytes (tamanho da página) do arquivo binário BACKING_STORE.bin.
- A posição de leitura deve ser calculada multiplicando o número da página por 256.
- Os dados lidos devem ser armazenados diretamente em uma matriz correspondente à memória física: `signed char main_memory[128][256]`.

Restrições de Implementação:

- Deve-se usar obrigatoriamente as funções nativas `fseek()` para posicionamento e `fread()` para a leitura do bloco de bytes em C.
- Incluir tratamento básico de erros para verificar se a leitura obteve sucesso.

Solicitação: Forneça laços de inicialização da memória, carregamento de página do arquivo binário e leitura de um byte específico adaptando ao template.

Prompt da Fase 2 - Passo 3: Gerenciamento da Tabela de Páginas

Contexto do Problema: Implementando a estrutura de dados e as funções de manipulação da Tabela de Páginas (Page Table) em C para o simulador de memória virtual.

Requisitos Funcionais:

- Criar uma tabela com 256 entradas utilizando um tipo estruturado para armazenar o número do quadro físico (*frame*), o bit de validade (*valid*), o contador de envelhecimento (*aging_counter*) e o bit de referência.
- Implementar função de inicialização para marcar todas as entradas como inválidas.
- Implementar função de busca (*lookup*) que retorna o quadro se a página for válida ou -1 se for inválida.
- Implementar funções para mapear (*update*) e desmapear (*invalidate*) páginas na tabela.

Restrições de Implementação:

- O tamanho do array deve respeitar as macros globais definidas no projeto (NUM_PAGES ou PAGE_TABLE_SIZE).

Solicitação: Gere o esqueleto completo do arquivo `page_table.c` contendo essas implementações de acordo com os requisitos.

Prompt da Fase 2 - Passo 4: Implementação da Escolha de Vítima (Aging Algorithm)

Contexto do Problema: Implementando o algoritmo de substituição de páginas por envelhecimento (*Aging*) no arquivo de gerência de memória simulada (`memory.c`).

Requisitos Funcionais:

- `select_victim_page`: Iterar por todas as entradas da tabela de páginas para identificar as páginas marcadas como válidas. Comparar seus contadores de envelhecimento de 8 bits e retornar o índice da página que possuir o menor valor acumulado.
- `handle_page_fault`: Quando a memória física estiver lotada, deve acionar a escolha da página vítima, recuperar o quadro físico em que ela residia, desmapeá-la da tabela de páginas através de invalidação, limpar sua respectiva entrada no TLB e reaproveitar esse mesmo quadro para carregar os novos dados vindos do arquivo binário.

Restrições de Implementação:

- Integrar-se com as assinaturas de funções utilitárias exportadas por `page_table.h` e `tlb.h`.

Solicitação: Modifique as funções de tratamento de falta de páginas e escolha de vítima no módulo de memória em C conforme os requisitos de paginação por demanda.

Prompt da Fase 2 - Passo 5: Cache TLB com Política FIFO Circular

Contexto do Problema: Criação do mecanismo de cache acelerador TLB com política de descarte baseada em envelhecimento de entrada FIFO (First-In, First-Out) em C.

Requisitos Funcionais:

- `tlb_lookup`: Percorrer o array de tamanho estático `TLB_SIZE` buscando por acertos de tradução de endereço.
- `tlb_insert`: Realizar a inserção de novos pares de mapeamento utilizando um contador circular sequencial `fifo_next` para sobrescrever a entrada mais antiga quando o cache atingir a capacidade máxima de 16 elementos.

Tratar casos de atualização de frames para chaves duplicadas.

- `tlb_remove`: Fornecer um mecanismo de busca e desligamento do bit de validade para páginas desalocadas pelo gerenciador global de memória física.

Restrições de Implementação:

- Utilizar aritmética modular (%) para o controle do índice circular de substituição.

Solicitação: Desenvolva o preenchimento de código limpo em C para os métodos de consulta, inserção FIFO e remoção do arquivo de gerenciamento do TLB.

Prompt da Fase 2 - Passo 6: Consolidação de Métricas Estatísticas

Contexto do Problema: Implementação do módulo de coleta e exibição de estatísticas de desempenho (`statistics.c`) para um simulador de paginação por demanda.

Requisitos Funcionais:

- `print_statistics`: Calcular a taxa de faltas de página (*Page Fault Rate*) e a taxa de acertos no cache (*TLB Hit Rate*) dividindo os respectivos contadores acumulados pelo total de endereços lógicos processados.
- Exibir os resultados formatados na tela com precisão de 3 casas decimais (`%.3f`).
- Prevenir erros de divisão por zero caso o contador de endereços esteja zerado.

Restrições de Implementação:

- Realizar a coerção explícita de tipos (*type casting*) de `int` para `double` ou `float` antes da divisão para evitar truncamento de inteiros na linguagem C.

Solicitação: Forneça o preenchimento de código completo para as funções de incremento de contadores e impressão do relatório final de estatísticas.